Multilevel AI Authenticity Engine

Comprehensive Project Analysis, Model Architecture & Structure Specification Report

System Name: Multilevel AI Authenticity Engine	**Core Framework:** PyTorch, OpenCV, Streamlit
Primary Application: AI Deepfake & Synthetic Media Detection	**Architecture Type:** Hierarchical Multi-Expert Ensemble
Models Used: Hybrid Dual CNN, MobileNetV2, ViT-B/16, GPT-4o-mini	**Input Modalities:** Images (JPG, PNG) & Videos (MP4, MOV, AVI)

1. Executive Overview & System Purpose

The **Multilevel AI Authenticity Engine** is a state-of-the-art forensic evaluation system engineered to detect synthetic media generated by modern generative artificial intelligence (including Generative Adversarial Networks like StyleGAN/ProGAN and Latent Diffusion Models like Stable Diffusion, Midjourney, and DALL-E).

Instead of relying on a single monolithic neural network—which is highly prone to overfitting, compression artifacts, and poor generalization across new generative models—this system employs a **hierarchical multi-level expert ensemble architecture**. Digital inputs undergo simultaneous forensic inspection across four distinct domain levels (Frequency, Facial, Semantic, Temporal) before an ensemble fusion engine aggregates probabilities into a definitive authenticity score.

2. Repository & System Structure

The codebase is cleanly modularized into frontend presentation pages, core decision & explanation engines, specialized model expert classes, and automated weight download handlers. Below is the complete repository layout:

Path / Module	Component Type	Description & Functionality
`app.py`	Main Entrypoint	Streamlit dashboard UI with custom CSS animations, cyber particle canvas, and main navigation cards.
`download_models.py`	Utility Script	Automated model weight downloader using requests and tqdm targeting GitHub Releases v1.0.
`requirements.txt`	Dependencies	Specifies required libraries: torch, torchvision, opencv-python, pillow, streamlit, openai, tqdm.
`src/core/ensemble_engine.py`	Core Logic	Central aggregation hub executing Level 1–3 experts, implementing weighted fusion and face override logic.
`src/core/explanation_engine.py`	Core Logic	Forensic reasoning engine leveraging OpenAI gpt-4o-mini with a structured local rule-based fallback.
`src/experts/level1_expert.py`	Expert Model	Level 1 Frequency Expert: Custom Dual-Branch Hybrid CNN (Spatial 2D Conv + 2D FFT Frequency Conv).
`src/experts/level2_expert.py`	Expert Model	Level 2 Face Integrity Expert: MobileNetV2 Deep CNN backbone + OpenCV Haar Cascade face detector.
`src/experts/level3_expert.py`	Expert Model	Level 3 Semantic Expert: Fine-tuned Vision Transformer (ViT-B/16) fine-tuning top 4 encoder layers.

`src/experts/level4_video_expert.py`	Expert Model	Level 4 Video Expert: Temporal frame sampling framework aggregating multi-frame ensemble predictions.
`pages/1_Image_Forensics.py`	UI Page	Single image evaluation page with confidence badges, progress bars, breakdown, and AI explanation.
`pages/2_Video_Forensics.py`	UI Page	Video analysis page executing frame extraction, per-frame evaluation, and average risk computation.
`pages/3_Batch_Analysis.py`	UI Page	Batch processing page for evaluating multiple images simultaneously with summary metrics.
`pages/4_System_Architecture.py`	UI Page	Interactive architecture documentation detailing model layers, weights, and decision rules.
`models/`	Model Store	Local directory storing downloaded PyTorch state dicts (.pth files) for Level 1, 2, and 3.

3. Deep-Dive into AI / Deep Learning Models Used

The core strength of the engine lies in its combination of diverse model families—ranging from signal processing CNNs to pre-trained transfer learning architectures, Transformer models, and Large Language Models. Below is the detailed technical specification of each model level:

LEVEL 1 — FREQUENCY & NOISE ARTIFACT EXPERT

Model Architecture: Custom Dual-Branch Hybrid Convolutional Neural Network (Spatial + Frequency FFT)
Model Class: HybridLevel1(nn.Module) in src/experts/level1_expert.py
Primary Task: Detect high-frequency spectral artifacts, grid noise patterns, upscaling boundaries, and compression signals invisible to human vision.

Architectural Mechanics:
• *Input Preprocessing:* Converts image to single-channel Grayscale, resizes to 32x32, transforms to Tensor.
• *Spatial Branch:* 3 sequential Conv2D blocks (1→32→64→128 channels with 3x3 kernels, padding=1, ReLU, 2x2 MaxPool2D). Output shape: (128, 4, 4).
• *Frequency Branch:* Computes 2D Fast Fourier Transform torch.fft.fft2(x), shifts zero-frequency to center via torch.fft.fftshift, calculates magnitude torch.abs(), and applies log scaling torch.log1p(). Passes through an identical 3-stage Conv2D block. Output shape: (128, 4, 4).
• *Fusion Classifier:* Concatenates spatial and frequency feature vectors (dim=1) yielding a combined 256-channel tensor. Flattens to 2048 units → Dense(256) → ReLU → Dropout(0.3) → Linear(256, 2) binary output.
• *Weights File:* models/level1/level1_hybrid.pth

LEVEL 2 — FACIAL INTEGRITY & DEEPFAKE EXPERT

Model Architecture: MobileNetV2 Deep CNN Backbone (Transfer Learning) + OpenCV Haar Cascade Detector
Model Class: Level2FaceExpert in src/experts/level2_expert.py
Primary Task: Detect facial synthesis seam artifacts, face-swapping boundaries, skin texture anomalies, and identity distortions in human faces.

Architectural Mechanics:
• *Face Detection Stage:* Employs OpenCV cv2.CascadeClassifier with haarcascade_frontalface_default.xml (scaleFactor=1.1, minNeighbors=5, minSize=40x40). Dynamically expands each bounding box by a **30% margin** (margin = int(0.3 * w)) to include jawline, hairline, and blending seams.
• *Classification Backbone:* PyTorch torchvision.models.mobilenet_v2 utilizing depthwise separable convolutions.
• *Custom Classifier Head:* Replaces standard head with Linear(last_channel, 512) → ReLU → Dropout(0.3) → Linear(512, 2).
• *Multi-Face Evaluation:* Converts crops to PIL RGB, resizes to 128x128, normalizes using ImageNet stats. Computes per-face fake probability, overall maximum fake probability, and average face score.
• *Weights File:* models/level2/level2_face_best.pth

LEVEL 3 — SEMANTIC CONSISTENCY EXPERT (VISION TRANSFORMER)

Model Architecture: Vision Transformer (ViT-B/16 - Fine-Tuned Encoder)
Model Class: Level3SemanticExpert in src/experts/level3_expert.py
Primary Task: Evaluate global structural realism, lighting consistency, shadow placement, anatomical correctness, and high-level scene context.

Architectural Mechanics:
• *Backbone Architecture:* Standard models.vit_b_16 pre-trained on ImageNet-1K (patch size 16x16, 12 transformer encoder blocks, 768 hidden dimension).
• *Fine-Tuning Strategy:* Freezes lower-level feature extraction layers (blocks 0–7) to retain universal visual representations. Unfreezes the top 4 Transformer Encoder blocks (model.encoder.layers[-4:]) for specialized fine-tuning on high-level semantic inconsistencies.
• *Classification Head:* Replaces standard 1000-class head with a 2-class linear projection layer nn.Linear(768, 2).
• *Transforms:* Uses PyTorch torchvision.transforms.v2 (Resize 224x224, float32 scaling, ImageNet normalization).
• *Weights File:* models/level3/level3_semantic_best.pth

LEVEL 4 — VIDEO TEMPORAL AGGREGATION EXPERT

Model Architecture: Frame-Level Temporal Sampling & Ensemble Aggregation Framework
Model Class: Level4VideoExpert in src/experts/level4_video_expert.py
Primary Task: Detect temporal flickers, frame-to-frame synthesis glitches, and deepfake video manipulations.

Architectural Mechanics:
• *Keyframe Extraction:* Reads video file via OpenCV cv2.VideoCapture. Uniformly samples 15 to 20 frames across the video duration using calculated frame steps step = total_frames // max_frames.
• *Hierarchical Evaluation:* Writes extracted frames to temporary storage and feeds each frame through the EnsembleEngine (executing Level 1, Level 2, Level 3 analysis).
• *Temporal Risk Metrics:* Computes maximum frame fake probability (peak anomaly) and mean frame fake probability across all keyframes.
• *Threshold Rules:* Max Fake > 0.75 → 'AI Generated', Avg Fake > 0.60 → 'Likely AI', Avg Fake < 0.35 → 'Authentic', else → 'Uncertain'.

LEVEL 5 — AI EXPLANATION ENGINE (LLM + RULE FALLBACK)

Model Architecture: Large Language Model Reasoning Layer (OpenAI gpt-4o-mini) + Deterministic Local Rule Fallback
Model Class: ExplanationEngine in src/core/explanation_engine.py
Primary Task: Generate human-readable forensic analysis reports explaining *why* the ensemble reached its specific verdict.

Architectural Mechanics:
• *API Integration:* Checks environment variable OPENAI_API_KEY. Constructs structured prompts containing Level 1, 2, and 3 output metrics and queries gpt-4o-mini (temperature=0.2).
• *Strict Guardrails:* Prompt explicitly instructs the LLM: *'Explain clearly WHY the system produced this verdict. Do NOT re-classify.'*
• *Local Fallback Engine:* If API key is missing or encounters rate limits/network failures, automatically falls back to _generate_local_explanation(), constructing a rule-driven bulleted summary of fake signals, face counts, and semantic metrics.

4. Ensemble Decision Logic & Mathematical Fusion

The EnsembleEngine in src/core/ensemble_engine.py aggregates predictions from all active experts to calculate the final Authenticity Score. The decision pipeline incorporates adaptive conditional weighting:

Condition / Scenario	Decision Mode	Mathematical Aggregation Formula
Face Detected & Level 2 Fake > 0.80	Face Override	$P_{final} = P_{L2\_max\_fake}$ (Overrides other models due to critical deepfake facial signal)
Face Detected & Level 2 Fake ≤ 0.80	Weighted Fusion (Face Present)	$P_{final} = 0.20 \cdot P_{L1} + 0.50 \cdot P_{L2\_max} + 0.30 \cdot P_{L3}$ (Prioritizes facial expert while factoring frequency & semantics)
No Face Detected in Image	Weighted Fusion (No Face)	$P_{final} = 0.50 \cdot P_{L1} + 0.50 \cdot P_{L3}$ (Equal 50/50 split between Frequency and Semantic ViT experts)

Final Verdict Classification Thresholds:

- **AI Generated:** Final AI Probability $P_{final} > 0.70$ (70% threshold)
- **Authentic:** Final AI Probability $P_{final} < 0.35$ (35% threshold)
- **Uncertain:** Final AI Probability between 0.35 and 0.70 (requires human forensic review)

5. User Interface & Page Workflow Breakdown

The user interface is powered by Streamlit and features a modern, responsive dark theme with CSS keyframe gradient animations and an interactive JavaScript particle canvas rendered on the background HTML5 canvas element.

Page Module	Interactive Features & Capabilities
`app.py (Home)`	Main portal featuring glowing navigation cards for Image Forensics, Video Forensics, and Batch Analysis, styled with CSS hover scaling and drop-shadow effects.
`1_Image_Forensics.py`	Single image file uploader (JPG, PNG). Renders AI Probability metric, interactive progress bar, dynamic confidence badges (High >80%, Moderate >50%, Low ≤50%), multi-level expert probability breakdowns, and LLM text explanation.
`2_Video_Forensics.py`	Video uploader (MP4, MOV, AVI). Plays uploaded video, extracts keyframes to temp directory, runs multi-level analysis frame-by-frame, displays average AI percentage, and presents final video verdict.
`3_Batch_Analysis.py`	Bulk image uploader accepting multiple files. Renders summary metrics (Total Processed, Real Count, AI Count, Uncertain Count, Average AI Probability) and individual card breakdowns with progress bars.
`4_System_Architecture.py`	Technical sitemap rendering system overview, detailed descriptions of Level 1–5 experts, ensemble decision rules, and architectural motivation.

6. Comprehensive Summary Table of All AI Models

Expert Level	Model Type / Family	Framework	Key Technique	Output Type
Level 1	Hybrid Dual ConvNet	PyTorch	2D FFT Log-Magnitude + Spatial Conv	Probabilities (Real/Fake)
Level 2	MobileNetV2 + OpenCV	PyTorch / OpenCV	Haar Cascade + 30% Margin Crop	Per-face & Max Fake Prob
Level 3	Vision Transformer (ViT-B/16)	PyTorch / Torchvision	Unfrozen Top 4 Encoder Layers	Semantic Fake Prob & Conf
Level 4	Temporal Frame Aggregator	OpenCV / PyTorch	Uniform Frame Step Sampling	Video AI Percentage & Verdict
Level 5	LLM / Rule Engine	OpenAI / Python	GPT-4o-mini + Rule Fallback	Natural Language Text Report